

Evaluating IPAD under generator shift

An evaluation protocol and accompanying software

Hongxi Pu | 3 October 2026 | Version 0.1

Abstract

A text detector may perform well on one language model and fail on another. This protocol sets out a small, matched evaluation of IPAD under generator shift. The proposed experiment holds the writing domain, source prompts and detector configuration fixed while changing the model that produces the AI text. It measures discrimination, classification performance and false positives on human text, with uncertainty estimated by resampling source groups. The accompanying software evaluates saved detector scores and has passed unit tests and numerical comparisons with an independent implementation. Data collection and IPAD inference remain to be completed; this version reports the experimental design and software checks, with no new detection results.

1. Research question

How does a fixed IPAD detector perform on a generator outside its original evaluation, when the domain and source prompts are held constant?

The study focuses on generator shift in a single writing domain. This keeps the initial comparison small enough to inspect individual errors and reconstructed prompts. Cross-domain transfer and paraphrasing can be studied in later experiments. The generator and corpus have not yet been selected. Candidate generators will be checked against the original paper and released evaluation data before being described as unseen.

2. Connection to IPAD

IPAD uses inverse prompts and consistency-based signals to detect AI-generated text [1]. This study will examine the transfer of a fixed IPAD configuration. Reconstructed prompts will also be inspected to understand what information the detector recovers from a passage and how that information changes across generators.

Detection performance and explanation quality are separate questions. A plausible reconstructed prompt may help describe a decision, but it does not by itself establish the source of a text or the faithfulness of an explanation. The initial study therefore treats these examples as qualitative observations.

Scope of this version

The completed work consists of this protocol, inspection of the public model resources and a saved-score evaluator. The evaluator has passed 12 unit tests and 200 numerical cross-checks. The remaining experimental work is to select the data and generator, verify the inference pipeline and collect predictions.

Code and supporting files: github.com/p123hx/ipad-robustness.

Experimental design

3. Matched source groups

The proposed pilot uses 150 source groups from one corpus with documented provenance and suitable reuse terms. Each group consists of a source prompt and a human-written response. Fifty groups are assigned to source validation and 100 to held-out testing before text generation. All passages derived from the same prompt or source document remain in the same split. Exact and near-duplicate checks are applied before the split is finalized.

Condition	Human / AI records	Role
Source validation: 50 groups	50 / 50	Check source settings and select a threshold if calibration is needed.
Reference-generator test: 100 groups	100 / 100	Evaluate the fixed detector on the reference condition.
Shifted-generator test: the same 100 groups	100 / 100	Change the generator using matched prompts and human controls.

Reusing the 100 human test passages across the two conditions gives 500 evaluation rows from 400 distinct texts. Shared group identifiers preserve the pairing. Metrics are reported separately for each condition, so reused human passages are not counted twice in a pooled estimate. These counts define a pilot; a formal power analysis has not been conducted.

4. Fixed detector configuration

The base model, adapters, tokenizer, prompt templates, numerical precision, maximum lengths and score aggregation are fixed across test conditions. The regeneration model and its settings are also held constant. The decision threshold is taken from a documented original setting or selected using source validation alone, before shifted-test scores are inspected.

The experiment records generator versions, sampling parameters, generation dates, corpus revisions, source identifiers and text hashes. Any change to a published inference setting is recorded with the resulting configuration.

5. Scores and failure records

The evaluator expects a continuous score in $[0,1]$, with higher values indicating AI-generated text. Score direction, normalization and yes/no token handling must be verified in the inference pipeline. Component scores, reconstructed prompts and regenerated passages are retained with their sample identifiers.

A manifest records every attempted sample and any inference failure. Missing predictions are resolved or reported with the achieved coverage. They are not silently excluded from the evaluation.

6. Qualitative inspection

A small set of source groups will be selected before reviewing the detector outputs. For both generator conditions, the analysis will examine prompt relevance and the consistency signals used by the detector. These observations will be reported alongside errors, including cases where a convincing reconstruction accompanies an incorrect classification.

Evaluation and software checks

7. Metrics and uncertainty

The evaluator reads `sample_id`, `group_id`, `split`, `condition`, `label` and `score` from a CSV file. For each test condition, it computes accuracy, precision, recall, F1, human-text false-positive rate and AUROC. A score greater than or equal to the supplied threshold is classified as AI-generated. AUROC assigns average ranks to tied scores; undefined metrics are returned as null.

Confidence intervals use a cluster bootstrap over source groups. All rows belonging to a sampled group are retained together. The software reports 95% percentile intervals, the random seed and the number of usable replicates. It withholds intervals when too few replicates are valid. This version estimates each condition separately; paired intervals for performance differences are left for a later extension.

8. Software validation

Check	Outcome
Unit tests	12 passed, covering ranking and ties, threshold boundaries, undefined metrics, duplicate identifiers, split leakage, invalid scores, repeatability and required study metadata.
Independent metric comparison	AUROC, accuracy and F1 agreed with scikit-learn 1.8.0 across 200 artificial cases, with absolute tolerance 1e-12.
Example execution	The complete evaluator ran on 8 numerical test rows with 2,000 group-bootstrap draws.

The test inputs are artificial labels and scores. They check the evaluator, not the IPAD detector, and contain no human-written or model-generated passages. The repository keeps these outputs under `software-validation` to distinguish them from future experimental results.

Running the checks

```
python3 -m unittest discover -s tests -v

python3 evaluate.py --predictions examples/fixture_predictions.csv
--metadata examples/fixture_metadata.json --threshold 0.5
--purpose software-validation --bootstrap 2000
--out validation/fixture_metrics.json
```

The second command is shown across lines for readability; the README provides a copyable shell command. The threshold 0.5 is a test setting. Evaluation of real predictions requires a separately documented threshold and completed study metadata. The evaluator uses Python 3.10 or later and the standard library.

Validation limits

Input checks detect missing scores, duplicate row identifiers, declared split leakage and mismatches with the expected row count. They cannot verify human authorship, identify every duplicate passage or establish the absence of training overlap. Those checks depend on the corpus and inference records.

Implementation notes and outlook

9. Reproducing the inference pipeline

The public IPAD model repository provides adapters and test resources [2], linked from the code repository [3]. It uses Phi-3-medium-128k-instruct as the base model [4]. The score evaluator is independent of this inference stack and does not load model weights.

Source inspection identified several details to resolve before inference. The model-card comments and paper appear to use the PTCV and RC names differently. The two helper script names also do not clearly describe their contents, and probability extraction uses a local file produced by a customized inference engine. The adapter mapping, training formats and sample-to-score correspondence will therefore be checked on a small set of known inputs before collecting the test predictions. These observations concern reproduction details; they do not assess the published results.

10. Planned analysis

After selecting a corpus and a new generator, the next step is to verify the full pipeline, including reconstruction, regeneration and score combination. The source groups, generation settings and decision threshold will then be fixed for the held-out evaluation.

The analysis will compare performance across the matched generator conditions and examine individual errors. Because the human controls and detector are shared, their false-positive rate should be unchanged under deterministic scoring; any difference would prompt a check of the execution settings. Changes in AI-text recall and AUROC will help identify whether the detector transfers to the new generator.

Later experiments may extend the design to a second domain or to paraphrased text. Uncertainty-aware decisions and more systematic evaluation of reconstructed prompts are further directions. Their usefulness will depend on the errors observed in the initial study.

References

[1] Chen et al. IPAD: Inverse Prompt for AI Detection - A Robust and Interpretable LLM-Generated Text Detector. NeurIPS 2025. [Proceedings record](#).

[2] IPAD model resources. huggingface.co/bella/c/IPAD.
Revision inspected: e57952ab8f36a77421a02f96ac2a2fa13d324b6e.

[3] IPAD code repository. github.com/Bella/c/IPAD-Inver-Prompt-for-AI-Detection.
Accessed 3 October 2026.

[4] Phi-3-medium-128k-instruct. [Microsoft model repository](#).
Revision inspected: a088b37c71d441ab6d862bb3fcfe6165b3014702.

Acknowledgment: AI tools assisted with drafting and software development.